Other Outer Joins

- Left outer join:
  - Include the left tuple even if there's no match
- Right outer join:
  - Include the right tuple even if there's no match
- Full outer join:
  - Include the both left and right tuples even if there's no match

How many entries will output table have?

- Left table has L entries
- Right table has R entries

- Inner join:
  - Minimum number of entries: 0
  - Maximum number of entries: L*R

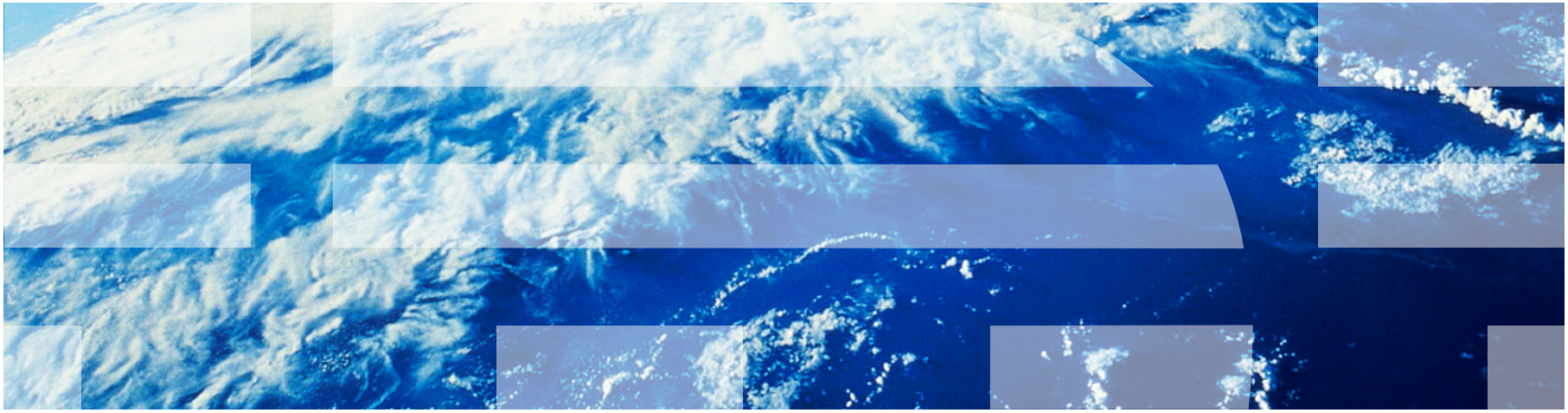
- Left outer join:
  - Minimum number of entries: L
  - Maximum number of entries: L*R

- Right outer join:
  - Minimum number of entries: R
  - Maximum number of entries: L*R

- Full outer join:
  - Minimum number of entries: L+R
  - Maximum number of entries: L*R

Borrowed from Shiva Shivakumar and Theodoros Rekatsinas

Nested Queries



SQL is Compositional

Can construct powerful query chains (e.g., f(g(...(x)))

Inputs / outputs are multisets

⇒ Output of one query can be input to another (nesting)!

⇒ Including on same table

Nested queries: Sub-queries Return Relations

Company(name, city)
Product(name, manufacturer)
Purchase(id, product, buyer)

```
SELECT   Product.manufacturer
FROM     Purchase, Product
WHERE    Purchase.product = Product.name
         AND Purchase.buyer = 'Alice''
```

1. Companies making products bought by 'Alice'
2. Location of companies?

Nested queries: Sub-queries Return Relations

Company(name, city)
Product(name, manufacturer)
Purchase(id, product, buyer)

```
SELECT Company.city
FROM   Company
WHERE  Company.name  IN (
       SELECT  Product.manufacturer
       FROM    Purchase, Product
       WHERE   Purchase.product = Product.name
               AND Purchase.buyer = 'Alice')
```

1. Companies making products bought by 'Alice'
2. Location of companies?

Subqueries Return Relations

You can also use operations of the form:

- s > ALL R
- s < ANY R
- EXISTS R

Example:

Product(name, price, category, maker)

```
SELECT name
FROM   Product
WHERE  price > ALL(
    SELECT price
    FROM   Product
    WHERE  maker = 'Gizmo-Works')
```

Find products that are more expensive than all those produced by "Gizmo-Works"

```
SELECT p1.name
FROM   Product AS p1
WHERE  p1.maker = 'Gizmo-Works'
  AND EXISTS(
        SELECT p2.name
        FROM   Product AS p2
        WHERE  p2.maker <> 'Gizmo-Works'
                 AND p1.name = p2.name)
```

<> means !=

Find 'copycat' products, i.e. products made by competitors with the same names as products made by "Gizmo-Works"

Note the scoping of the variables!

Example: Complex Correlated Query

Product(name, price, category, maker, year)

```
SELECT DISTINCT  x.name, x.maker
FROM             Product AS x
WHERE            x.price > ALL(
        SELECT y.price
        FROM     Product AS y
        WHERE  x.maker = y.maker
AND y.year < 1972)
```

Find products (and their manufacturers) that are more expensive than all products made by the same manufacturer before 1972

Can be very powerful (also much harder to optimize)

Aggregates inside nested queries

1. Aggregates inside nested queries. Remember SQL is **compositional**
2. Hint 1: Break down query description to steps (subproblems)
3. Hint 2: Whenever in doubt always go back to the definition

Aggregates inside nested queries: example

Example:

"Using a *single SQL query*, find all of the stations that had the highest daily precipitation (across all stations) on any given day."

Precipitation

station_id	day	precipitation
122	1	33
122	4	20
351	1	10
191	7	45

```
SELECT station_id, day
FROM   Precipitation,
         (SELECT day AS maxd, MAX(precipitation) AS maxp
          FROM precipitation
          GROUP BY day)
WHERE day = maxd AND precipitation = maxp
```

Step 1

```
[redacted]
        (SELECT day AS maxd, MAX(precipitation) AS maxp
         FROM precipitation
         GROUP BY day)
[redacted]
```

maxd	maxp
1	33
4	20
7	45

Step 2

```
SELECT station_id, day
FROM   precipitation,
         (SELECT day AS maxd, MAX(precipitation) AS maxp
          FROM precipitation
          GROUP BY day)
WHERE day = maxd AND precipitation = maxp
```

station_id	day	precipitation
122	1	33
122	4	20
351	1	10
191	7	45

JOIN

maxd	maxp
1	33
4	20
7	45

station_id	day
122	1
122	4
191	7